Listing 3.6 `useChat` example

```
import { useChat } from 'ai/react';

export default function Chat() {
  const { messages, input, handleInputChange, handleSubmit} =
    useChat({ api: '/api' });

return (
    <div>
      <Messages data={messages} />.
      <form onSubmit={handleSubmit}>
        <input
          value={input}
          placeholder="Ask me something..."
          onChange={handleInputChange}
        />
      </form>
    </div>
 )
  }
```

This line imports the useChat hook from the ai/react library.

This line calls the useChat hook, initiating a chat session with the LLM. It then uses destructuring to extract several values from the object returned by the hook, including the list of messages and the form submission handler.

This is responsible for displaying the conversation history. It receives the messages array as data.

Compared to the custom `useChatFormSubmit` hook, there are many similarities, and it's almost a drop-in replacement. The `useChat` hook accepts an `api` parameter that will be used to point to the fetch endpoint, which returns to the client. It similarly returns the list of messages and the current value of the prompt. It also provides the form submit handler that we use on a form field. However, behind the scenes, this hook also manages streamed responses in an efficient way without having us do the extra work of implementing them from scratch.

By using the `useChat` or `useCompletion` hooks, we can significantly simplify the implementation of streaming-based conversational interfaces and text completion features in our React application. These hooks handle the complexities of managing state, updating the UI, and integrating with the AI provider, allowing you to focus on building engaging user experiences.

3.2.3 Integrating streaming into Astra AI

Let's now bring everything together and enable streaming into our Astra AI project. Based on the previous section, we can identify what we need to modify to enable streaming support.

> **NOTE** The source code for the following example is available at https://github.com/Generative-AI-Web-Apps/Code/tree/main/ch03/chat-streaming.

1. *Replace the* `generateText` *function with the* `streamText` *function*—As mentioned, the `streamText` function from the `ai` package is suitable for generating streams of text generations from a language model.
2. *Return a streaming text response instance*—Instead of returning the AI stream response object as is, we need to convert it to a `StreamingTextResponse` that is suitable for